Enterprise Web Application Assignment 4

Collections Framework & Generics: UCS553

Q1. Course Registration Portal

Design a comprehensive course registration portal combining multiple collection types for optimal data handling, prerequisites tracking, and waitlist management. Use a HashMap <String, Course> for fast course lookups by course code. Each Course object must contain a HashSet of unique enrolled student IDs, an ArrayList for ordered prerequisite codes, and a LinkedList acting as a first-come, first-served waitlist queue. Handle registration requests with capacity checks and automated waitlist promotion logic.

1. Question1.java

```
package q1;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.HashSet;
import java.util.LinkedList;

class Course {
    private String courseCode;
    private String title;
    private int capacity;
    private HashSet<String> enrolledStudents;
    private ArrayList<String> prerequisites;
    private LinkedList<String> waitlist;

    public Course(String courseCode, String title, int capacity) {
        this.courseCode = courseCode;
        this.title = title;
        this.capacity = capacity;
        this.enrolledStudents = new HashSet<>();
        this.prerequisites = new ArrayList<>();
        this.waitlist = new LinkedList<>();
    }

    public String getCourseCode() {
        return courseCode;
    }

    public String getTitle() {
        return title;
    }

    public int getCapacity() {
        return capacity;
    }

    public HashSet<String> getEnrolledStudents() {
        return enrolledStudents;
    }

    public ArrayList<String> getPrerequisites() {
        return prerequisites;
    }

    public LinkedList<String> getWaitlist() {
        return waitlist;
    }

    public void addPrerequisite(String prereqCode) {
        prerequisites.add(prereqCode);
    }

    public boolean isFull() {
```

```
        return enrolledStudents.size() >= capacity;
    }

    public boolean enrollStudent(String studentId) {
        return enrolledStudents.add(studentId);
    }

    public void addToWaitlist(String studentId) {
        if (!waitlist.contains(studentId)) {
            waitlist.add(studentId);
        }
    }

    public boolean dropStudent(String studentId) {
        return enrolledStudents.remove(studentId);
    }

    public String promoteWaitlistStudent() {
        if (!waitlist.isEmpty()) {
            String nextStudentId = waitlist.removeFirst();
            enrolledStudents.add(nextStudentId);
            return nextStudentId;
        }
        return null;
    }
}

class Student {
    private String studentId;
    private String name;
    private HashSet<String> completedCourses;

    public Student(String studentId, String name) {
        this.studentId = studentId;
        this.name = name;
        this.completedCourses = new HashSet<>();
    }

    public String getStudentId() {
        return studentId;
    }

    public String getName() {
        return name;
    }

    public HashSet<String> getCompletedCourses() {
        return completedCourses;
    }

    public void addCompletedCourse(String courseCode) {
        completedCourses.add(courseCode);
    }

    public boolean hasCompleted(String courseCode) {
        return completedCourses.contains(courseCode);
    }
}

class CourseRegistrationPortal {
    private HashMap<String, Course> courseMap;
    private HashMap<String, Student> studentMap;

    public CourseRegistrationPortal() {
        courseMap = new HashMap<>();
        studentMap = new HashMap<>();
    }

    public void addCourse(Course course) {
        courseMap.put(course.getCourseCode(), course);
    }
```

```
    public void addStudent(Student student) {
        studentMap.put(student.getStudentId(), student);
    }

    public void registerStudent(String studentId, String courseCode) {
        Course course = courseMap.get(courseCode);
        Student student = studentMap.get(studentId);

        if (course == null) {
            System.out.println("Error: Course " + courseCode + " not found.");
            return;
        }
        if (student == null) {
            System.out.println("Error: Student " + studentId + " not found.");
            return;
        }

        if (course.getEnrolledStudents().contains(studentId)) {
            System.out.println("Student " + student.getName() + " (" + studentId + ") is already enroll
ed in " + courseCode + ".");
            return;
        }

        for (String prereq : course.getPrerequisites()) {
            if (!student.hasCompleted(prereq)) {
                System.out.println("Registration failed for " + student.getName() + ": Missing prerequi
site " + prereq + " for " + courseCode + ".");
                return;
            }
        }

        if (!course.isFull()) {
            course.enrollStudent(studentId);
            System.out.println("Enrollment successful: " + student.getName() + " enrolled in " + course
Code + ".");
        } else {
            course.addToWaitlist(studentId);
            int pos = course.getWaitlist().indexOf(studentId) + 1;
            System.out.println("Course " + courseCode + " is full. " + student.getName() + " added to w
aitlist at position " + pos + ".");
        }
    }

    public void dropStudent(String studentId, String courseCode) {
        Course course = courseMap.get(courseCode);
        Student student = studentMap.get(studentId);

        if (course == null || student == null) {
            System.out.println("Error: Invalid course or student ID.");
            return;
        }

        if (course.dropStudent(studentId)) {
            System.out.println(student.getName() + " dropped " + courseCode + ".");
            String promotedId = course.promoteWaitlistStudent();
            if (promotedId != null) {
                Student promotedStudent = studentMap.get(promotedId);
                String pName = promotedStudent != null ? promotedStudent.getName() : promotedId;
                System.out.println("Waitlist promotion: " + pName + " (" + promotedId + ") is automatic
ally enrolled in " + courseCode + ".");
            }
        } else if (course.getWaitlist().remove(studentId)) {
            System.out.println(student.getName() + " removed from " + courseCode + " waitlist.");
        } else {
            System.out.println("Student " + studentId + " is not registered in " + courseCode + ".");
        }
    }

    public void displayCourse(String courseCode) {
        Course course = courseMap.get(courseCode);
```

```
        if (course == null) {
            System.out.println("Course not found.");
            return;
        }
        System.out.println("\nCourse Details: " + course.getCourseCode() + ": " + course.getTitle());
        System.out.println("Capacity: " + course.getCapacity());
        System.out.println("Prerequisites: " + course.getPrerequisites());
        System.out.println("Enrolled Students (" + course.getEnrolledStudents().size() + "/" + course.g
etCapacity() + "): " + course.getEnrolledStudents());
        System.out.println("Waitlist Queue: " + course.getWaitlist());
    }
}

public class Question1 {
    public static void main(String[] args) {
        System.out.println("=== Course Registration Portal ===");

        CourseRegistrationPortal portal = new CourseRegistrationPortal();

        Course c1 = new Course("CS101", "Introduction to Programming", 3);
        Course c2 = new Course("CS201", "Data Structures", 2);
        c2.addPrerequisite("CS101");

        portal.addCourse(c1);
        portal.addCourse(c2);

        Student s1 = new Student("S101", "Shreyaas");
        s1.addCompletedCourse("CS101");

        Student s2 = new Student("S102", "Rahul");
        s2.addCompletedCourse("CS101");

        Student s3 = new Student("S103", "Aman");

        Student s4 = new Student("S104", "Pooja");
        s4.addCompletedCourse("CS101");

        Student s5 = new Student("S105", "Rohan");
        s5.addCompletedCourse("CS101");

        portal.addStudent(s1);
        portal.addStudent(s2);
        portal.addStudent(s3);
        portal.addStudent(s4);
        portal.addStudent(s5);

        System.out.println("\n1. Registering S101 (Shreyaas) for CS201:");
        portal.registerStudent("S101", "CS201");

        System.out.println("\n2. Registering S103 (Aman) for CS201 (without prerequisite):");
        portal.registerStudent("S103", "CS201");

        System.out.println("\n3. Registering S102 (Rahul) for CS201:");
        portal.registerStudent("S102", "CS201");

        System.out.println("\n4. Registering S104 (Pooja) for CS201 (Course Full):");
        portal.registerStudent("S104", "CS201");

        System.out.println("\n5. Registering S105 (Rohan) for CS201 (Course Full):");
        portal.registerStudent("S105", "CS201");

        portal.displayCourse("CS201");

        System.out.println("\n6. S102 (Rahul) drops CS201:");
        portal.dropStudent("S102", "CS201");

        portal.displayCourse("CS201");
    }
}
```

Q1 Output

Terminal: Q1 Output

```
shreyaas@shreyaass-MacBook-Air Enterprise Web Applications % /opt/homebrew/opt/openjdk/bin/java -cp
bin q1.Question1
=== Course Registration Portal ===

1. Registering S101 (Shreyaas) for CS201:
Enrollment successful: Shreyaas enrolled in CS201.

2. Registering S103 (Aman) for CS201 (without prerequisite):
Registration failed for Aman: Missing prerequisite CS101 for CS201.

3. Registering S102 (Rahul) for CS201:
Enrollment successful: Rahul enrolled in CS201.

4. Registering S104 (Pooja) for CS201 (Course Full):
Course CS201 is full. Pooja added to waitlist at position 1.

5. Registering S105 (Rohan) for CS201 (Course Full):
Course CS201 is full. Rohan added to waitlist at position 2.

Course Details: CS201: Data Structures
Capacity: 2
Prerequisites: [CS101]
Enrolled Students (2/2): [S102, S101]
Waitlist Queue: [S104, S105]

6. S102 (Rahul) drops CS201:
Rahul dropped CS201.
Waitlist promotion: Pooja (S104) is automatically enrolled in CS201.

Course Details: CS201: Data Structures
Capacity: 2
Prerequisites: [CS101]
Enrolled Students (2/2): [S104, S101]
Waitlist Queue: [S105]
```

Q2. Media Player Queue Engine

Build a media player queue engine that supports fast insertions at arbitrary positions, recent history tracking, and bidirectional navigation. Utilize a LinkedList<Song> to manage playback queues. Implement playlist features that allow adding songs to the front (priority queueing), removing duplicate song entries efficiently, and using List Iterator to traverse the playlist both forward and backward without index-based performance bottlenecks.

1. Question2.java

```
package q2;

import java.util.HashSet;
import java.util.Iterator;
import java.util.LinkedList;
import java.util.ListIterator;
import java.util.Objects;

class Song {
    private String title;
    private String artist;
    private String duration;

    public Song(String title, String artist, String duration) {
        this.title = title;
        this.artist = artist;
        this.duration = duration;
    }

    public String getTitle() {
        return title;
    }

    public String getArtist() {
        return artist;
    }

    public String getDuration() {
        return duration;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (obj == null || getClass() != obj.getClass()) return false;
        Song song = (Song) obj;
        return title.equalsIgnoreCase(song.title) && artist.equalsIgnoreCase(song.artist);
    }

    @Override
    public int hashCode() {
        return Objects.hash(title.toLowerCase(), artist.toLowerCase());
    }

    @Override
    public String toString() {
        return title + " by " + artist + " (" + duration + ")";
    }
}

class MediaPlayerQueue {
    private LinkedList<Song> queue;
    private LinkedList<Song> history;

    public MediaPlayerQueue() {
        queue = new LinkedList<>();
        history = new LinkedList<>();
    }
```

```
    public void addSong(Song song) {
        queue.add(song);
        System.out.println("Added to queue: " + song);
    }

    public void addSongToFront(Song song) {
        queue.addFirst(song);
        System.out.println("Priority added to front: " + song);
    }

    public void insertSong(int index, Song song) {
        if (index >= 0 && index <= queue.size()) {
            queue.add(index, song);
            System.out.println("Inserted at position " + index + ": " + song);
        } else {
            System.out.println("Invalid position for insertion: " + index);
        }
    }

    public void removeDuplicates() {
        HashSet<Song> seen = new HashSet<>();
        Iterator<Song> it = queue.iterator();
        int count = 0;
        while (it.hasNext()) {
            Song s = it.next();
            if (seen.contains(s)) {
                it.remove();
                count++;
            } else {
                seen.add(s);
            }
        }
        System.out.println("Removed " + count + " duplicate song(s) from queue.");
    }

    public void playNext() {
        if (queue.isEmpty()) {
            System.out.println("Queue is empty. No song to play.");
            return;
        }
        Song current = queue.removeFirst();
        history.add(current);
        System.out.println("Now Playing: " + current);
    }

    public void displayQueue() {
        System.out.println("\nCurrent Playback Queue (" + queue.size() + " songs):");
        if (queue.isEmpty()) {
            System.out.println("  (Empty)");
            return;
        }
        int idx = 1;
        for (Song s : queue) {
            System.out.println("  " + idx + ". " + s);
            idx++;
        }
    }

    public void displayHistory() {
        System.out.println("\nRecently Played History (" + history.size() + " songs):");
        if (history.isEmpty()) {
            System.out.println("  (Empty)");
            return;
        }
        int idx = 1;
        for (Song s : history) {
            System.out.println("  " + idx + ". " + s);
            idx++;
        }
    }
```

```
    public void traverseBidirectional() {
        System.out.println("\nBidirectional Traversal using ListIterator:");
        ListIterator<Song> it = queue.listIterator();

        System.out.println("Forward traversal:");
        while (it.hasNext()) {
            int nextIndex = it.nextIndex();
            Song s = it.next();
            System.out.println("  [" + nextIndex + "] Forward: " + s);
        }

        System.out.println("\nBackward traversal:");
        while (it.hasPrevious()) {
            int prevIndex = it.previousIndex();
            Song s = it.previous();
            System.out.println("  [" + prevIndex + "] Backward: " + s);
        }
    }
}

public class Question2 {
    public static void main(String[] args) {
        System.out.println("=== Media Player Queue Engine ===");

        MediaPlayerQueue player = new MediaPlayerQueue();

        Song s1 = new Song("Starboy", "The Weeknd", "3:50");
        Song s2 = new Song("Shape of You", "Ed Sheeran", "3:53");
        Song s3 = new Song("Blinding Lights", "The Weeknd", "3:20");
        Song s4 = new Song("Believer", "Imagine Dragons", "3:24");
        Song s5 = new Song("Shape of You", "Ed Sheeran", "3:53"); // duplicate
        Song s6 = new Song("Starboy", "The Weeknd", "3:50");     // duplicate
        Song prioritySong = new Song("Bohemian Rhapsody", "Queen", "5:55");

        System.out.println("\n1. Adding songs to queue:");
        player.addSong(s1);
        player.addSong(s2);
        player.addSong(s3);

        System.out.println("\n2. Priority queueing (Adding to front):");
        player.addSongToFront(prioritySong);

        System.out.println("\n3. Inserting song at arbitrary position (index 2):");
        player.insertSong(2, s4);

        System.out.println("\n4. Adding duplicates to queue:");
        player.addSong(s5);
        player.addSong(s6);

        player.displayQueue();

        System.out.println("\n5. Removing duplicate songs using HashSet:");
        player.removeDuplicates();

        player.displayQueue();

        System.out.println("\n6. Playing songs and tracking history:");
        player.playNext();
        player.playNext();

        player.displayHistory();
        player.displayQueue();

        System.out.println("\n7. Bidirectional navigation over remaining queue:");
        player.traverseBidirectional();
    }
}
```

Q2 Output

Terminal: Q2 Output

```
shreyaas@shreyaass-MacBook-Air Enterprise Web Applications % /opt/homebrew/opt/openjdk/bin/java -cp
bin q2.Question2
=== Media Player Queue Engine ===

1. Adding songs to queue:
Added to queue: Starboy by The Weeknd (3:50)
Added to queue: Shape of You by Ed Sheeran (3:53)
Added to queue: Blinding Lights by The Weeknd (3:20)

2. Priority queueing (Adding to front):
Priority added to front: Bohemian Rhapsody by Queen (5:55)

3. Inserting song at arbitrary position (index 2):
Inserted at position 2: Believer by Imagine Dragons (3:24)

4. Adding duplicates to queue:
Added to queue: Shape of You by Ed Sheeran (3:53)
Added to queue: Starboy by The Weeknd (3:50)

Current Playback Queue (7 songs):
  1. Bohemian Rhapsody by Queen (5:55)
  2. Starboy by The Weeknd (3:50)
  3. Believer by Imagine Dragons (3:24)
  4. Shape of You by Ed Sheeran (3:53)
  5. Blinding Lights by The Weeknd (3:20)
  6. Shape of You by Ed Sheeran (3:53)
  7. Starboy by The Weeknd (3:50)

5. Removing duplicate songs using HashSet:
Removed 2 duplicate song(s) from queue.

Current Playback Queue (5 songs):
  1. Bohemian Rhapsody by Queen (5:55)
  2. Starboy by The Weeknd (3:50)
  3. Believer by Imagine Dragons (3:24)
  4. Shape of You by Ed Sheeran (3:53)
  5. Blinding Lights by The Weeknd (3:20)

6. Playing songs and tracking history:
Now Playing: Bohemian Rhapsody by Queen (5:55)
Now Playing: Starboy by The Weeknd (3:50)

Recently Played History (2 songs):
  1. Bohemian Rhapsody by Queen (5:55)
  2. Starboy by The Weeknd (3:50)

Current Playback Queue (3 songs):
  1. Believer by Imagine Dragons (3:24)
  2. Shape of You by Ed Sheeran (3:53)
  3. Blinding Lights by The Weeknd (3:20)

7. Bidirectional navigation over remaining queue:

Bidirectional Traversal using ListIterator:
Forward traversal:
  [0] Forward: Believer by Imagine Dragons (3:24)
  [1] Forward: Shape of You by Ed Sheeran (3:53)
  [2] Forward: Blinding Lights by The Weeknd (3:20)

Backward traversal:
  [2] Backward: Blinding Lights by The Weeknd (3:20)
  [1] Backward: Shape of You by Ed Sheeran (3:53)
  [0] Backward: Believer by Imagine Dragons (3:24)
```